application, it is infrequent and does not significantly affect the overall execution speed.

In comparison, the execution speed of most DSP algorithms is limited almost completely by the number of multiplications and additions required. For example, Fig. 28-2 shows the implementation of an FIR digital filter, the most common DSP technique. Using the standard notation, the input signal is referred to by $x[\ ]$, while the output signal is denoted by $y[\ ]$. Our task is to calculate the sample at location n in the output signal, i.e., $y[n]$. An FIR filter performs this calculation by multiplying appropriate samples from the input signal by a group of coefficients, denoted by: a_0, a_1, a_2, a_3,$\cdots$, and then adding the products. In equation form, $y[n]$ is found by:

$$y[n] = a_0x[n] + a_1x[n-1] + a_2x[n-2] + a_3x[n-3] + a_4x[n-4] + \cdots$$

This is simply saying that the input signal has been *convolved* with a filter kernel (i.e., an impulse response) consisting of: a_0, a_1, a_2, a_3,$\cdots$. Depending on the application, there may only be a few coefficients in the filter kernel, or many thousands. While there is some data transfer and inequality evaluation in this algorithm, such as to keep track of the intermediate results and control the loops, the math operations dominate the execution time.

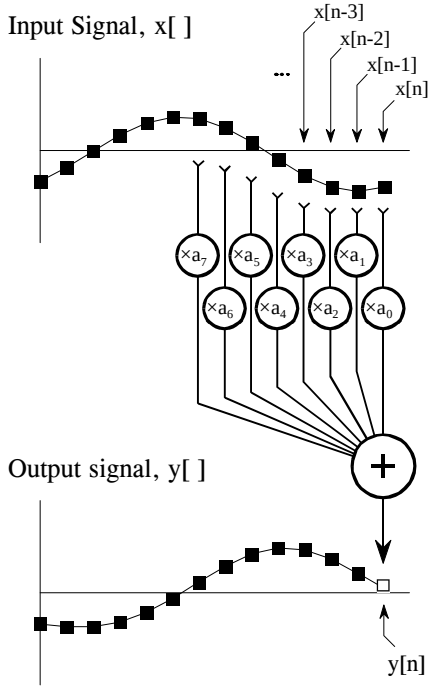

FIGURE 28-2
FIR digital filter. In FIR filtering, each sample in the output signal, y[n], is found by multiplying samples from the input signal, x[n], x[n-1], x[n-2], ..., by the filter kernel coefficients, a_0, a_1, a_2, a_3 ..., and summing the products.